
Does Fault Localization Beat a Fresh Attempt? A Placebo-Controlled Study of Test-Guided Code Repair

Anik Jha
Independent Researcher
anik.k.jha@gmail.com

Abstract

Fault localization can focus a code model’s repair on the statements a failing test implicates, but a targeted edit may succeed merely because it is small, and a second model call may succeed without using the failure at all. We separate these explanations with three arms applied to the same failed candidate (blind whole-solution resampling, spectrum-based localization followed by suspect-span infilling, and same-length infilling at a disjoint random code span) across three frozen 26–32B models, three benchmarks and 488 failing candidates, plus a separately declared 24B fourth model from a third family. Three results follow. First, localization is rarely available: only 9.0% of failing candidates expose a failing public test with a usable spectrum, because the largest group of failures passes every test the user can see. Second, among the 177 candidates localizable from a strong suite, localized infilling loses decisively to blind resampling at a matched number of attempts (3:40, $p = 3.0 \times 10^{-9}$), in the direction opposite to our hypothesis. That loss replicates in a third family: on 62 candidates from Mistral-Small-24B, run on two of the three benchmarks, it is -11.3 points (0.6% against 11.9%, 95% CI [-16.6, -6.8], discordance 0:25), the largest margin we measure, so our rule’s two-family replication clause is satisfied only in the inverted direction, by the method losing to its own comparator. Third, giving the span arm more room does not rescue it: widening the edit until the median span grows from 1 line to 5 lines leaves it still losing, and an adaptive policy adds at most +0.6 points over always editing wider. Against the random-span placebo localized infilling leads pooled (11:1, Holm-adjusted $p = .019$), but that lead resolves in no individual model under the per-model attempt-level analysis our shipped analysis plan designates primary (best Holm $p = .087$), so we report the location effect as suggestive rather than established. Re-pricing attempts as tokens narrows but does not overturn this: a span attempt spends 21.7 generated tokens against 371.1, yet 16 localized attempts reach 6.8% while one blind attempt already reaches 10.1%. The mechanism is visible in the generations: infilling reproduces the removed span verbatim in 48.9% of attempts and yields 0.23 distinct programs per attempt against 0.83 for resampling. A capability probe down to 0.6B cannot carry the location comparison at all, because 65.5% of the span arms’ spliced programs do not parse at that scale, so we restrict every localization conclusion to the 24–32B models tested.

1 Introduction

Testing and debugging are explicitly part of verifiable coding: tests provide executable evidence that a behavior is wrong, while fault localization proposes where to intervene. Large language models (LLMs) make the intervention easy to generate, but they also introduce a basic evaluation problem. If

a model repairs a localized span, did the test signal identify useful code, did any small edit have a better chance than regenerating the whole program, or did an additional sample provide all of the benefit?

These explanations require different controls. Recent placebo-controlled studies of frozen small code models find that blind resampling can match or outperform feedback-conditioned self-repair [Iskan, 2026a,b, Verma, 2026]. In parallel, fault-localization studies show that more precise locations can improve repair when locations are supplied by developer patches or other oracles [Sepidband et al., 2026, Liu et al., 2026]. Neither result answers whether *predicted*, test-derived localization makes a targeted retry preferable to a fresh attempt. Refining the whole program has been argued to beat localized hunks [Xu et al., 2024], and the closest infilling work varies masked locations [Koutchene et al., 2023], but neither separates the fresh sample from the edit size with a matched attempt budget and a random-location control.

We test this missing comparison on the same initially failing candidate with three arms: blind whole-solution resampling; Ochiai spectrum-based fault localization (SBFL) from public tests, followed by fill-in-the-middle (FIM) replacement of the most suspicious tied span; and a disjoint, same-length random-code-span placebo. This design separates two questions:

1. Does the chosen location add value over editing an equally sized but unrelated code region?
2. Does localized editing add value over spending the same number of attempts on fresh solutions?

We declared a demanding success rule before running the experiment, recorded in the project’s kickoff documentation: localized infilling had to beat both controls, exceed the placebo by at least five percentage points with Holm-adjusted $p < .05$, and replicate across at least two model families. We call this rule *pre-specified* rather than *preregistered* because the study plan was not registered with an external service. The rule was not met: the placebo margin is +3.2 points rather than five, no per-model test of the placebo comparison survives correction under either analysis, and localized infilling loses to blind resampling instead of beating it. Its replication clause is satisfied only in that last, inverted sense. We report what the evidence does support instead.

What it supports is a four-part answer. Localization is usually unavailable: only 9.0% of failing candidates expose a failing public test with a usable spectrum. Where it is available, a fresh attempt beats a targeted one at a matched attempt count (3:40, $p = 3.0 \times 10^{-9}$), resolving in two of the three original checkpoints under both analyses and again in a fourth model from a third family (-11.3 points, 95% CI [-16.6, -6.8]), so the cross-family bar we set in advance is met by the method losing to its own control. Localized infilling does lead an unrelated edit of the same size when pairs are pooled (11:1, Holm $p = .019$), but in no individual model under the primary analysis, so we carry it as suggestive. Widening the edit does not rescue it, and re-pricing attempts as tokens narrows rather than overturns the result: localization leads only below 372 tokens, the price of one fresh attempt.

Our contribution is evaluative rather than a new repair algorithm. We provide (1) a two-control protocol for test-guided repair, in which the random-span placebo is what separates “the location was right” from “the edit was small”; (2) a localizability audit over 488 failures reporting how often public tests support localization at all; (3) a budget analysis in both attempts and tokens, with the edit-diversity measurements that explain why the two disagree; and (4) five concrete safeguards (Appendix B), each of which silently changed a measured result in our own pipeline before it was fixed. We make no formal-verification or correctness-guarantee claim.

2 Related work

Feedback and blind resampling. Iskan [2026a,b] use preregistered placebo-controlled decompositions of self-repair in frozen small code models. Verma [2026] finds blind retry to be a particularly strong baseline below 7B parameters. These studies regenerate an entire solution. We retain their blind control but ask whether restricting the edit to a test-localized span changes the comparison at larger model scale.

Localization for LLM repair. Locations in this literature come from oracles or from predictors. On the oracle side, using 500 SWE-bench Verified instances and ground-truth patch locations,

Sepidband et al. [2026] find that file localization is crucial and that adding line context can help or hurt depending on how it is combined with broader context, and Townsend et al. [2026] feed ground-truth locations to Agentless at three granularities on SWE-Bench-Mini, finding function level highest but task dependent. On the predictor side, SHERLOC performs structured, agentic repository localization and measures downstream repair [Tamoyan et al., 2026], Sepidband et al. [2026b] make the localizer reason over a repository exceeding the context window, and Al Awad and Ivanov [2026] runs the closest contrast to ours at file granularity, where pooled resolved rate rises from 44.7% to 48.9–49.1% with predicted locations and to 52.4% with gold ones. None of these, so far as we could verify, reports an untargeted-regeneration arm at a matched attempt budget or a random-location control, so localization gains and retry-alone gains remain unseparated in the published record; and the setting is complementary rather than overlapping, file-level selection inside a repository against single-function repair, where regenerating everything is the control that matters most. Liu et al. [2026] vary localization precision across two APR techniques and three LLMs, and report that imprecise fault localization makes repair harder and widens the gap between techniques. Fan et al. [2023] match our setting most closely, repairing failing Codex solutions under unguided, line-guided and statement-guided Codex editing and finding guided repair “nearly comparable” to unguided; their unguided arm still edits the failed program rather than drawing a fresh sample, and they run no random-location control.

Targeted intervention against re-running. Luan et al. [2026] ask our question in a different unit, whether multi-agent repair methods “causally repair ... or merely stochastically repair by leveraging the randomness of LLM sampling”. Replaying a trajectory to an anchor and regenerating downstream, they find symptom-driven intervention repairs 20.15% of 536 annotated failures against 6.90% for unguided rerunning, the opposite sign to ours. Their anchor replays a prefix reconstructed from recorded logs and still regenerates a whole downstream trajectory, where our span keeps a fixed prefix and suffix around a gap of a few tokens. Zhu et al. [2026] measure the complementary thing, that frontier models “often regenerate correct but over-edited solutions”, proposing edit-level precision; our size-matched placebo is the control-side analogue.

Infilling and patch structure. Xia et al. [2022] study pre-trained models for repair at scale, reporting that suffix context, which an infilling formulation adds, yields more fixes at a higher compilation rate. Their complete-function setting “makes no assumption of ... the location of the bug” while their infilling and single-line settings are given it; it counts patches per setting rather than at a matched attempt budget, with no random-location arm and 2022-era models. Our no-op measurement is a possible cost of that suffix conditioning, which our design does not isolate. Xu et al. [2024] argue our result’s direction as a design principle: restricting an LLM to statement-level hunks “hinders LLMs from exploring potential patches beyond the given locations”. Theirs is a prompting method with no random-location arm and no matched attempt budget, so it motivates our question but does not answer it. Koutchene et al. [2023] evaluate InCoder by masking up to 50 candidate spans and sampling repairs, demonstrating that generative infilling is a viable repair primitive. The closest published contrast in representation space is Silva et al. [2025], who fine-tune LoRA adapters and compare a whole-function input against inputs carrying fault-localization comments or an infilling mask, and a full fixed function output against the fixed chunk alone. They “only assume that one is able to identify a code region that encompasses all the buggy lines”, varying how a supplied location is presented to a *fine-tuned* model; we vary whether a test-derived location is used at all, for frozen models, against a fresh sample at a matched attempt count and a random-location control. Ochiai SBFL is already combined with LLM repair: Farzandway and Ghassemi [2025] rank lines by Ochiai score into a chain-of-thought loop on Codeflaws, passing localization as prompt context rather than using it to constrain the edit region. Multi-hunk defects become harder as patch divergence and dispersion grow [Nashid et al., 2025], so our single contiguous-span setting is a deliberately limited first test. Classic SBFL assigns line suspiciousness from coverage spectra [Abreu et al., 2007]; we use Ochiai and avoid picking an arbitrary line when several tie at the maximum.

3 Experimental design

3.1 Study population and localization

For each model and benchmark, we first greedily generate one candidate per task. A *dead* candidate fails the full benchmark tests. We attempt localization only when at least one test in the localizing

suite fails and executing those tests yields line coverage. Thus all three-arm analyses are conditional on the same localizable subset; resampling outcomes from nonlocalizable dead candidates are never included.

We localize under two suites, because which failures a repair loop can even see turns out to dominate everything else it does. The *public* signal uses only the tests a deployed loop has, namely HumanEval’s and MBPP’s original cases. The *strong* signal additionally reads the augmented suite (every base test plus up to 80 sampled augmented ones), which no deployed loop has; we report it as an upper bound on available localization signal and never as a method. The strong signal is also what makes the comparison affordable to power: it takes the localizable population from 44 candidates to 177. Because the three-arm comparison conditions on localizable failures, we separately replay this decision for every dead candidate and record why it succeeds or fails (Figure 1).

For source line ℓ , Ochiai suspiciousness is

$$s(\ell) = \frac{n_f(\ell)}{\sqrt{N_f [n_f(\ell) + n_p(\ell)]}}, \quad (1)$$

where N_f is the number of failing tests in the localizing suite and $n_f(\ell)$ and $n_p(\ell)$ count failing and passing tests that execute ℓ . We take the contiguous range spanning all lines tied at the maximum score. Candidates are excluded if this range exceeds 60% of source lines or if no disjoint same-length code-only placebo span exists. Under the public signal the augmented tests are used only for final scoring; under the strong signal they also inform the spectrum, which is exactly why that condition is a bound and not a method.

3.2 Three arms

Every arm receives $K = 16$ stochastic samples with temperature 0.8 and nucleus probability 0.95. The arms differ as follows.

- **Blind resample:** generate a fresh complete solution from the original problem, without the failed candidate, coverage, or test result.
- **Localized infill:** replace the maximum-suspiciousness span while conditioning on its unchanged prefix and suffix, using native FIM tokens where present and a marked gap in a chat prompt otherwise. Decoding stops at the model’s own FIM terminators, read from its tokenizer rather than assumed, and the completion is cut back to the middle it was asked for, since an instruct model handed a raw FIM prompt keeps going into a test harness and splicing that overrun makes the file unparseable. The cuts are structural (a markdown fence, a dedent out of the span’s block, or a verbatim suffix) and identical for this arm and the placebo, so they cannot favour either (§5).
- **Random-span placebo:** replace a disjoint span of the same line length, sampled only from executable code lines rather than comments, docstrings, or blanks. It uses the same native or prompted infilling path as the localized arm.

The budget is matched in *number of attempts*. Span arms allow 256 new tokens and whole-solution resampling 512 on HumanEval+ and MBPP+ and 1536 on LiveCodeBench, whose tasks are whole classes rather than single functions and which truncated badly at a lower ceiling. The ceilings are far above what either arm typically uses on its own benchmark, so they are not what drives the token gap we report; the arms’ measured output lengths are. Because attempts and tokens are different currencies, we report the comparison in both, and treat neither as the definitive one.

3.3 Models, benchmarks, and scoring

We evaluate Qwen2.5-Coder-32B-Instruct-AWQ, Qwen3.6-27B and Gemma-4-26B-A4B-it, and add Mistral-Small-3.2-24B-Instruct as a cross-family replication. The first uses 4-bit AWQ weights, the others local unquantized checkpoints. Both Qwen tokenizers expose FIM tokens; Gemma and Mistral use prompted pseudo-FIM. Mistral is a declared independent replication with the same estimator rather than a fourth cell of the three-model analysis: adding a model to a correction family after that family has been analysed and its plan published would change the multiplicity structure of an already-reported result. It ran on HumanEval+ and MBPP+ only, so its 62 candidates are not part of

the 488 the three-model analysis draws on. Thinking is disabled through Qwen3.6’s chat-template option so a reasoning preamble does not consume the code budget. All weights are frozen; there is no training or adaptation.

The evaluated models are the object of study rather than tools used to conduct it. Separately, and disclosed under the workshop’s LLM policy: the harness, analysis scripts, and manuscript text were written with substantial AI coding assistance under author direction. The experimental design, the pre-specified decision rule, the choice of controls, and every claim in this paper are the authors’ responsibility, and each reported number is regenerated from the result files by a checked script rather than transcribed.

HumanEval+ v0.1.10 and MBPP+ v0.2.0 supply both the base tests and the augmented ones [Liu et al., 2023]. A third benchmark, the function-call subset of LiveCodeBench [Jain et al., 2024] filtered to problems dated on or after 2024-07-01, contributes harder and more recent problems; we do not call it contamination-free, because we did not verify any checkpoint’s training cutoff, and §5 reports why that caveat is load-bearing there. An arm *unlocks* a candidate–task pair if any of its 16 completions passes every available test. Each EvalPlus test invocation is bounded by EvalPlus’s own rule, the larger of one second and four times the reference solution’s runtime on that input, rather than by a flat constant (§5); LiveCodeBench, which has no reference timing, uses a flat five-second bound. Our scorer adopts that timeout rule but compares outputs to the reference by exact equality rather than reusing EvalPlus’s per-task tolerance and order-insensitivity logic, so our absolute rates are not interchangeable with published EvalPlus numbers; every arm is scored identically, so it cannot favour an arm. On LiveCodeBench a baseline generation that hit the token ceiling is excluded from the dead population rather than counted as a failure, since truncation is a harness outcome and not a model one; that removes 0, 16 and 17 candidates respectively, which is why the dead counts below are smaller than the raw pass rates in §5 imply. Experiments ran on one NVIDIA DGX Spark with 128GB unified memory; generated-token counts are measured per attempt, but wall-clock durations were not systematically recorded.

A capability probe, not a fourth model. Because every arm conditions on failures, a model’s worth to this design is its failure count, not its benchmark score, and that count falls sharply as capability rises: on LiveCodeBench the three models yield 122, 46, 13 dead candidates in order of increasing pass rate. We therefore probed *downward*, on Qwen3-0.6B, 1.7B and 4B over HumanEval+ and MBPP+ (Appendix A).

3.4 Inference

The kickoff decision rule (Appendix E) is written in terms of *unlocks*: whether any of K samples passes. The Wave 1B analysis plan, specified before its generations, made the *per-attempt* success rate primary and the unlock test secondary, on the ground that collapsing 16 samples to one bit discards most of the data. We report both outcomes and label their distinct roles.

Both analyses are per model (Table 1), pooling each model’s tasks across benchmarks and Holm-correcting the three tests within each comparison family. The primary one takes the difference in per-attempt success rate with a task-clustered bootstrap: a task’s 16 samples are correlated, so tasks are the resampled unit and both arms are drawn together to preserve the pairing. The secondary one uses a two-sided exact McNemar/binomial test on discordant unlock outcomes. Pooled task–model-pair tests (Table 4) are sensitivity summaries under both plans: different models are not exchangeable replicates. We also report each arm’s $\text{unlock}@k$ curve under the standard unbiased estimator. All use the same generations, and the released outcome files retain every attempt rather than only one bit per task.

All three arms come from a single pass of one harness on the same candidates, so no comparison here splices runs made under different decoding rules.

4 Results

4.1 How often can test-guided repair fire at all?

Figure 1 replays the localization decision on all 488 dead candidates. Public tests support localization for 44 of them (9.0%). The dominant loss is not a weak localizer but an absent signal: 212 candidates

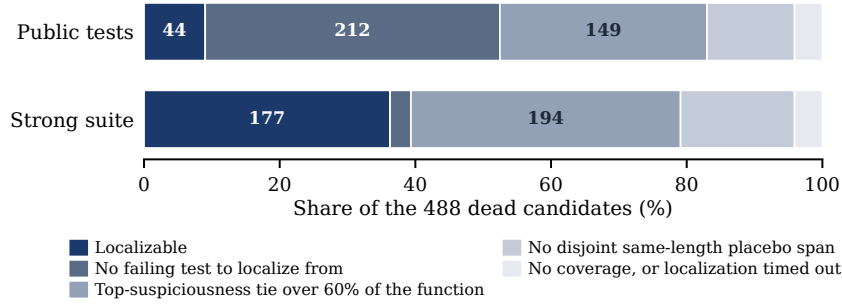


Figure 1: Where 488 dead candidates go (three models across HumanEval+, MBPP+ and Live-CodeBench). The dark segment is the localizable population the span arms can run on. Under public tests the dominant loss is an absent signal; under the strong suite, which no deployed loop has, it becomes a tied top-suspiciousness block too diffuse to edit.

pass every public test and fail only hidden ones, so no spectrum exists to compute. Localizing from the augmented suite instead raises coverage to 177 (36.3%) and nearly eliminates that category, at which point the binding constraint becomes structure rather than tests: the maximum-suspiciousness tie spans more than 60% of the function for most of the rest, because straight-line statements co-execute under every test. Where localization does apply the span is small: a median of 0.07 of the file under the public signal and 0.06 under the strong one.

Two consequences matter beyond this study. On this population a deployed repair loop that waits for a visible failing test could act on only 9.0% of failures, and the ones it does see are a biased sample: the strong signal adds 62 candidates whose bugs survive every public test. What bounds test-guided repair here is the strength of the available verifier rather than the quality of the localizer.

4.2 Does the location matter, and does it beat starting over?

Table 1 gives the primary per-model analysis and Table 4 the pooled sensitivity summary. Take the placebo comparison first, because it is the one the placebo exists to make: does an edit at the implicated location do something an equally sized edit at an unrelated one does not?

Under the primary analysis it does not resolve. The per-attempt difference is positive in every model, but it does not resolve in any model after Holm correction (best Holm $p = .087$), and the secondary per-model unlock test resolves in none of the three either. Only the pooled summary crosses the threshold: localized infilling unlocks tasks the placebo does not far more often than the reverse (11:1, exact $p = .006$, Holm-adjusted $p = .019$), at 4.3% of attempts against 1.1%, a difference of +3.2 points (95% CI [+0.5, +6.2]). That pooling treats three models as exchangeable replicates, which both of our plans decline to do for a confirmatory claim, so the honest statement is that the location plausibly matters and this experiment does not establish it.

Against blind resampling at a matched number of attempts the ordering reverses, the margin is far larger (3:40, $p = 3.0 \times 10^{-9}$; 10.1% of blind attempts succeed against 4.4% of localized ones), and this is the one result that resolves within individual models under *both* analyses. The primary attempt-level test survives Holm correction in 2 of the three checkpoints, and the secondary unlock test survives it in the same two (Table 1); the third resolves under neither, on too few discordant pairs to separate a null from a small effect. Our pre-specified rule demanded replication across at least two model *families*. Both resolving checkpoints are Qwen, which alone would not clear that bar, so we ran a fourth model from a third family. Mistral-Small-24B reproduces the effect on 62 candidates at -11.3 points (0.6% of localized attempts succeed against 11.9% of blind ones, 95% CI [-16.6, -6.8]), the largest margin in the study; its unlock discordance is 0:25 ($p = 6.0 \times 10^{-8}$), meaning resampling unlocked twenty-five candidates localization did not and localization unlocked none that resampling did not. The bar is therefore met, by the negative result. Table 1 collects all four.

One caveat we checked rather than waved through. Mistral’s spliced programs fail to parse in 19.9% of attempts against 2.5% for the others, so part of that margin could be an interface failure rather than a localization failure. Restricted to attempts that do parse, the difference is -12.2 points (95%

Table 1: Per-model tests on the strong-signal population under both designated analyses. Primary (Wave 1B plan, shipped in the supplement): per-attempt success rate, task-clustered bootstrap. Secondary (kickoff rule): the unlock outcome, exact McNemar, discordance localized-only : comparator-only. Holm correction runs across the three models of the pre-specified family within each comparison and analysis; bold marks Holm $p < .05$. Δpp is localized minus comparator. [†]Mistral-Small-24B is a declared independent replication with the same estimator, so it carries no adjusted p : it is not a member of that correction family.

Model	N	Primary: per attempt Δpp (95% CI)	Holm p	Secondary: unlock Disc.	Holm p
<i>vs. random placebo</i>					
Qwen2.5-Coder-32B	91	+1.3 [+0.1, +3.5]	.087	4:0	.375
Qwen3.6-27B	52	+2.9 [-3.1, +9.2]	.382	4:1	.500
Gemma-4-26B-A4B	34	+8.8 [+0.0, +20.6]	.156	3:0	.500
Mistral-Small-24B [†]	62	+0.6 [+0.0, +1.7]	—	2:0	—
<i>vs. blind resample</i>					
Qwen2.5-Coder-32B	91	-7.1 [-12.3, -2.3]	.016	1:19	< .001
Qwen3.6-27B	51	-9.1 [-16.4, -2.2]	.022	1:19	< .001
Gemma-4-26B-A4B	34	+2.9 [-2.2, +10.3]	.435	1:2	1.000
Mistral-Small-24B [†]	62	-11.3 [-16.6, -6.8]	—	0:25	—

CI [-17.8, -7.2]): it widens rather than narrows, so it is not an artifact of malformed infills. Mistral also regenerates the removed span verbatim in only 26.0% of attempts, about half the pooled rate of the others, so it explores more and still loses by more. Against its own placebo it behaves like every other model, +0.6 points ($p = .258$), which does not resolve.

The one model that does not reproduce the effect, Gemma-4-26B-A4B, is both the only non-Qwen of the original three and the only mixture-of-experts model on the roster; the three dense models all reproduce it. With a single MoE model we cannot separate those explanations and do not try to.

We report the deployable condition separately and it does not resolve: restricted to the 44 candidates localizable from public tests alone, the placebo comparison is 7:1 ($p = .070$) and the resampling comparison 2:10 ($p = .039$), neither surviving correction. Every claim that resolves in this study therefore depends on a localization signal a deployed loop would not have.

Halving the budget leaves both effects in place: re-run on the first 8 samples of the same generations, which varies the budget and nothing else, the placebo discordance is 10:1 ($p = .012$, +3.1 points against +3.2 at full budget) and the resampling discordance 4:34 ($p = 6.0 \times 10^{-7}$). Because both budgets read the same generations this is a sensitivity check showing directional stability, not an independent replication.

4.3 Attempts are the wrong unit of cost

The arms do not spend the same compute per attempt: a span replacement averages 21.7 generated tokens against 371.1 for a whole solution, a factor of 17.1. Per attempt, blind resampling overtakes localized infilling by the third sample and keeps climbing. Re-pricing in tokens (Figure 2) moves the crossover but does not remove it. Below 372 generated tokens, the cost of one whole-solution attempt, the blind arm cannot buy a single attempt while the span arm has already reached 6.8%; at and above that price the blind arm leads at every budget we can evaluate, its first attempt reaching 10.1% against the 6.8% that 16 localized attempts reach in total, and 27.8% by the last. This normalizes by each arm’s measured mean token cost rather than capping tokens directly, so it ignores prompt and prefill cost, which favours the span arms whose prompt carries the whole file. It also pools cells whose costs differ by an order of magnitude: mean resampling length runs from 119 tokens on the cheapest model–benchmark cell to 1213 on the dearest, the latter being whole-class LiveCodeBench tasks. 372 is therefore a pooled threshold for this population and not a constant a deployment can plan against; the ordering it summarises is robust, the number is not. A directly token-capped, wall-clock-matched comparison is the right way to settle it and we did not run one.

So the honest reading is narrow. Localization is what a budget too small for one fresh attempt can afford, which is a real regime for a latency-bound or per-token-billed deployment, and it is not an

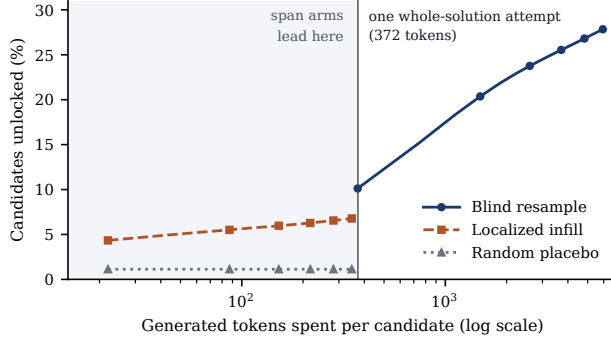


Figure 2: Unlock rate against the generated tokens it costs, strong-signal population, $k = 1$ to 16 attempts per arm priced at each arm’s measured mean token cost. Localized infilling leads only left of the line, where the blind arm cannot afford a single attempt; past it the blind arm’s first attempt already exceeds what 16 localized attempts reach in total, and the span arms are flat.

efficiency advantage that survives into any larger budget. What both views agree on is that the placebo never moves: extra budget spent at an unrelated location buys nothing at all.

4.4 Why the span arms saturate, and whether more room helps

Infilling conditioned on a fixed prefix and suffix frequently regenerates the removed span character for character: 48.9% of localized attempts and 41.4% of placebo attempts leave the program unchanged and cannot unlock anything, against 10.2% for blind resampling. Distinct programs per attempt say the same thing: resampling 0.83, the placebo 0.31, localized infilling fewest at 0.23 (Table 3). This is the infilling analogue of the anchoring reported for feedback-conditioned repair [Iscan, 2026a, Verma, 2026], with the surrounding code as the anchor rather than the failure message, and it binds hardest in the *localized* arm: the maximum-suspiciousness span is by construction the one the model just committed to and that its prefix and suffix most strongly imply, so it is the span it is most likely to write again.

The wins are concentrated too, which is why the placebo line in Figure 2 is flat rather than shallow. Counting a task as saturated when an arm unlocks it on all but at most one of its 16 attempts, the placebo saturates 2 of the 2 tasks it unlocks at all, localized infilling 6 of 12, and resampling 2 of 49. The span arms’ wins are the cases where the replacement is already determined, so extra samples have nothing to find.

That invites the reading that the negative result is an artifact of degenerate attempts. It is not. Scoring only attempts that changed the program narrows the gap without closing it (8.5% against blind resampling’s 11.3%, the placebo 1.9%), and rejecting repeats does not rescue it either: collapsing each task’s attempts to its 663 distinct middles lifts the localized rate only to 5.6%. The limit is the proposal distribution, not the duplicates drawn from it.

So we changed the proposal distribution. On 155 candidates where both arms stay valid at a larger size we widened the edit by two lines each side, moving the median span from 1 line to 5 and cutting verbatim regeneration to under five per cent. Three variants ran, enumerated in Appendix C, and none of them rescues the method (Table 2). The adaptive policy does not beat static narrow editing in either correction-family model, and where it moves at all the movement is width rather than adaptivity: Qwen3.6’s +4.9 points over narrow editing is matched exactly by always editing wider, and adaptivity adds at most +0.6 points on top. The widened arm still loses to blind resampling.

A capability probe running the same arms on Qwen3-0.6B, 1.7B and 4B produces 256 localizable pairs and no usable *location* comparison: 65.5% of those models’ spliced programs do not parse. Appendix A reports why that is a boundary on the technique rather than a capability moderator, and what it does still establish.

5 Implementation findings and threats to validity

Five defects in our own pipeline each changed a measured result before it was fixed, and each is a general risk for this kind of study rather than an artifact of our setup. Appendix B catalogues them: an unconstrained placebo span that could land entirely in a docstring, a single-line `argmax` selecting by ordering rather than evidence, a flat per-test timeout indefensible in both directions, an unregistered FIM terminator that spliced unparseable text into 13.4% of localized attempts, and a closed-fence extractor that silently returned whole truncated completions. They do not act in one direction: the placebo defect weakened a control, the span-selection and terminator defects weakened span arms, and the timeout and extractor defects changed which candidates were scored.

Multiplicity and what the tests carry. Holm correction across the 4 pooled comparisons leaves the resampling result at $p = 1.2 \times 10^{-8}$ and the placebo result at $p = .019$. Both of our plans locate the confirmatory analysis per model rather than pooled, and there only the resampling comparison resolves, in two of three checkpoints, under the primary attempt-level test and the secondary unlock test alike (Table 1). The placebo result is therefore pooled evidence with an interval attached and not a confirmatory finding; the replication clause is met by Mistral reproducing the loss; and the deployable public-test condition resolves nothing at all.

External validity. The analysis conditions on localizable failures from one greedy baseline per model, so it does not describe parse failures, hidden-only failures, or diffuse spectra, and all tasks are function or class level Python rather than repository repair or multi-hunk editing. Every model here without native FIM tokens is also non-Qwen, so we measured the interface rather than assuming it: re-running both Qwen models through the prompted path Gemma uses, on the same candidates and spans (Appendix F), leaves the resampling loss negative in both (-4.9 and -9.6 points), resolving in 1 of the two rather than 2. Prompt format is therefore not what explains Gemma, which is also the only mixture-of-experts checkpoint; nothing here separates family from architecture. The strong signal reads augmented tests a deployed loop would not have, and every result that resolves depends on it. On LiveCodeBench the pass-rate ordering tracks checkpoint recency rather than model size, which is what contamination would look like on a pool whose cutoffs we did not verify, so we claim none. Finally, test passage is bounded by the supplied suites and is not a proof of correctness.

6 Broader impact

A controlled baseline can stop organizations deploying a complicated repair loop where a fresh attempt does as well, cutting engineering and inference cost. Conversely, test-guided generation can produce plausible but incorrect or insecure code when suites are weak, and localization may add unwarranted confidence by making a patch look surgical; generated patches should remain sandboxed, reviewed, and tested against independent oracles. This work releases no new model or sensitive dataset.

7 Reproducibility

The workshop supplement holds the source, unit tests, aggregate results and spans used here. Every table, figure and headline quantity is emitted from the shipped result files rather than transcribed, the README maps each to its file, and the build reruns every documented command in a fresh extraction while tracing the emitter’s file reads, so a missing input fails the build. The paper build also rejects bare numerals and spelled-out magnitudes in prose, undefined references, unincluded fragments, and designations that disagree with the public analysis record. Weights and benchmarks are not redistributed; locks and durations were not preserved.

References

Mohammad Nour Al Awad and Sergey Ivanov. Loc2Repair: A framework for evaluating the impact of file-level issue localization in repo-level LLM repair. *arXiv:2606.30963*, 2026.

- Melika Sepidband, Hamed Taherkhani, Hung Viet Pham, and Hadi Hemmati. RGFL: Reasoning guided fault localization for automated program repair using large language models. [arXiv:2601.18044](#), 2026.
- Rui Abreu, Peter Zoetewij, and Arjan J. C. van Gemund. On the accuracy of spectrum-based fault localization. In *Testing: Academic and Industrial Conference Practice and Research Techniques*, 2007. doi:10.1109/taic.part.2007.13.
- Mehmet Iscan. Falsification, not exposure: An internally preregistered placebo-controlled decomposition of self-repair feedback in frozen small code models. [arXiv:2606.31511](#), 2026.
- Mehmet Iscan. Form, not content? A preregistered, placebo-controlled evaluation of learned error-conditioned self-repair through prompts and weights in frozen small code models. [arXiv:2607.12962](#), 2026.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. [arXiv:2403.07974](#), 2024.
- Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. Automated repair of programs from large language models. In *ICSE*, 2023. [arXiv:2205.10583](#).
- Charles Koutcheme, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. Automated program repair using generative models for code infilling. In *Artificial Intelligence in Education (AIED 2023)*, LNCS 13916, pages 798–803. Springer, 2023. doi:10.1007/978-3-031-36272-9_74
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *NeurIPS*, 2023. [arXiv:2305.01210](#).
- Junchi Liu, Ali Bigdeli, Roya Daneshi, Atu Ambala, Sudipto Ghosh, and Fabio Santos. Rethinking automated program repair: The impact of bug complexity, fault localization, and LLM cost-efficiency. In *ESEM*, 2026. [arXiv:2608.14065](#).
- Zhongwen Luan, Xiaoyu Zhang, Ming Hu, Yue Yang, Jiongchi Yu, and Xiaohong Chen. Repair or resample? Rethinking failure debugging in LLM multi-agent systems. [arXiv:2608.25920](#), 2026.
- Noor Nashid, Daniel Ding, Keheliya Gallaba, Ahmed E. Hassan, and Ali Mesbah. Characterizing multi-hunk patches: Divergence, proximity, and LLM repair challenges. [arXiv:2506.04418](#), 2025.
- Wang Bill Zhu, Miaosen Chai, Shangshang Wang, Yejia Liu, Song Bian, Honghua Dong, Willie Neiswanger, and Robin Jia. Precise debugging benchmark: Is your model debugging or regenerating? [arXiv:2604.17338](#), 2026.
- Melika Sepidband, Hung Viet Pham, and Hadi Hemmati. On the role of fault localization context for LLM-based program repair. [arXiv:2604.05481](#), 2026.
- Hovhannes Tamoyan, Sean Narenthiran, Erik Arakelyan, Mira Mezini, and Boris Ginsburg. SHER-LOC: Structured diagnostic localization for code repair agents. [arXiv:2606.24820](#), 2026.
- Joseph Townsend, Chandresh Pravin, Kwun Ho Ngan, and Matthieu Parizy. A study on the impact of fault localization granularity for repository-scale code repair tasks. [arXiv:2604.00167](#), 2026.
- Yuvraj Verma. Try again, don’t look back: Blind resampling outperforms self-repair in small code models. [arXiv:2607.26117](#), 2026.
- Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. Aligning the objective of LLM-based program repair. [arXiv:2404.08877](#), 2024.
- Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Practical program repair in the era of large pre-trained language models. [arXiv:2210.14179](#), 2022.
- André Silva, Sen Fang, and Martin Monperrus. RepairLLaMA: Efficient representations and fine-tuned adapters for program repair. In *IEEE Transactions on Software Engineering*, 2025. [arXiv:2312.15698](#).

A The capability probe

Running the same three arms on Qwen3-0.6B, 1.7B and 4B yields 256 localizable pairs, more than the three deployment-scale models produce together, and no usable comparison between the two span arms. Blind resampling unlocks 71 of 263 tasks while localized infilling unlocks 1 and the placebo 0. The reason is not that the location stops mattering: 65.5% of these models’ spliced programs do not parse at all, against 2.5% for the deployment-scale models. 34.5% of the small models’ splices are valid Python, so the arm is not wholly degenerate, but it is not valid often enough for the placebo contrast to carry a comparison.

We report this as a boundary on the technique and explicitly not as a capability moderator. A moderator claim would need the small models’ infills to be valid and merely unhelpful; here they are invalid, and the two explanations (that sub-4B models cannot use a localized gap, and that they cannot follow the raw fill-in-the-middle format at all) are not separated by this experiment. Gemma-4 already uses a prompted pseudo-FIM path for want of native FIM tokens (§3), and whether that path rescues small models is the experiment we did not run. What the probe does establish is that the blind-resampling result extends below 7B with a large margin, consistent with the small-model findings we build on [Iskan, 2026a, Verma, 2026].

B Implementation defects

Controls and localization. An unconstrained random span sometimes selected only a docstring or comment, which cannot change execution and is a structurally weak placebo; the final sampler masks comments, docstrings and blanks by tokenization and requires a disjoint span of identical line length. Straight-line statements that co-execute under every test receive identical spectra, so a single-line `argmax` selects by ordering rather than evidence; the final procedure spans the full contiguous maximum-score range and rejects it when too diffuse.

Untrusted code needs more than a clock. A flat five-second per-test bound proved indefensible in both directions: a correct HumanEval/39 solution needs about 5.1 seconds on our hardware, so the same generation passed or failed with machine load. EvalPlus scoring now follows EvalPlus’s own rule, the larger of one second and four times the reference solution’s runtime on that input. A timeout also does not bound memory: one candidate grew to 92GB resident and its worker was killed, deadlocking a shared process pool. Each scoring job now runs in its own supervised process with an address-space limit, so a hang or an out-of-memory candidate costs one task rather than the run.

Infilling needs its stop token, and truncation needs detection. Two extraction defects each turn a harness failure into an apparent model failure. The FIM terminator was not registered as a stop token and was stripped from the decoded text, so span generations ran past the intended middle and spliced unparseable text into 13.4% of localized attempts. Registering the terminators, read from each tokenizer rather than assumed, was necessary but not sufficient, because an instruct model handed a raw FIM prompt writes the replacement and then a demonstration harness. That left 5.2% of localized attempts unparseable, and the structural cuts of §3 bring that to 2.5%. Separately, extracting a fenced block with a closed-fence pattern silently returns the whole completion when generation was truncated before the closing fence, which misclassified 78 of 160 (Qwen3.6-27B) and 70 of 160 (Gemma-4-26B-A4B) LiveCodeBench generations as genuine failures. Raising the ceiling and detecting truncation leaves 16 and 17 generations still truncated in the reported run; those are dropped from the dead population rather than scored (§3), a population change a reader cannot infer from the pass rates. Both defects depress a treatment arm silently.

C Widening the localized edit

Table 2 is the falsification attempt described in §4.4, reported in full. Three arms ran against the narrow localized span on the 155 candidates where every variant stays valid at the larger size: always

Table 2: Widening the localized edit: per-attempt difference in points on the 155 candidates where both arms stay valid at the larger size. Bold marks Holm $p < .05$ within the pre-specified correction family. Rows two and three are the decomposition: width helps Qwen3.6, adaptivity adds nothing on top of width.

Comparison	Qwen2.5-Coder	Qwen3.6	Gemma-4
Adaptive vs static narrow	+0.5	+4.9	−6.2
Static wide vs static narrow	+0.4	+4.9	−6.9
Adaptive vs static wide	+0.1	+0.0	+0.6
Adaptive vs blind resample	−6.1	−5.5	−2.1

Table 3: What each arm produced on the strong-signal population, and what it produced that could count. Distinct/att. is unique spliced programs divided by attempts. Conditioning on a changed program removes the no-op difference from the comparison and does not reverse it.

Arm	Attempts	No-op	Distinct/att.	Unparsed	Success/att.	Success changed
Blind resample	2815	10.2%	0.83	0.5%	10.12%	11.27%
Localized infill	2832	48.9%	0.23	2.5%	4.34%	8.51%
Random placebo	2832	41.4%	0.31	2.5%	1.13%	1.93%

editing two lines wider on each side, an adaptive policy that widens only the 80.6% of attempts whose narrow output was a no-op or a duplicate, and a placebo held to the same realized edit size and generation-call count. Registering the static-wide arm in advance is what makes the table readable: Qwen3.6’s +4.9 points over narrow editing reads as evidence for adaptive localization until the static-wide column shows the same margin, at which point the movement is width alone and the adaptive policy contributes at most +0.6 points on top of it. The bottom row is the one that matters for the paper’s claim: every widened variant still loses to blind resampling.

D Generation integrity and pooled sensitivity

Two tables sit behind claims the main text states in one line each. Table 3 is the arm-level accounting: how many attempts each arm spent, how often it returned the program it was given, how many distinct programs it produced, how often the splice failed to parse, and its success rate both unconditionally and restricted to attempts that changed something. The last column is the answer to the obvious objection that the negative result is an artifact of degenerate attempts: restricting to attempts that changed the program narrows the gap and does not close or reverse it. All three arms are measured on the same strong-signal population, so the rates are directly comparable; the resampling row carries slightly fewer attempts because one localizable candidate has no resampling arm.

Table 4 is the pooled sensitivity summary. It is not the primary analysis and is not offered as one: both of this project’s planning documents locate the confirmatory test per model, which is Table 1. Its value is that it shows the two localization signals side by side, and the public-signal rows are the deployable condition. Neither public-signal comparison resolves, which is the result a reader planning a deployment should take from this paper.

E Pre-specified decision rule and artifact map

The decision rule below was recorded in the study plan before any results existed. Because the plan was not registered with an external service, we call it pre-specified rather than preregistered. The win bar required localized infilling to beat placebo by at least five percentage points with Holm-adjusted $p < .05$, beat blind resampling, and replicate across at least two model families. A localized-placebo tie or loss killed the localization mechanism claim; a failure to beat resampling killed the practical recommendation. All three win clauses failed: the observed placebo margin is +3.2 points rather than five, no per-model placebo test survives correction under either analysis, and localized infilling loses to blind resampling rather than beating it. The replication clause is the only one satisfied, and only in the inverted direction, because what replicates is the loss: it holds across two Qwen checkpoints

Table 4: Both comparisons, both signals. Discordance is localized-only : comparator-only over paired candidates; p is a two-sided exact McNemar test on those pairs. Attempt-level rates treat each of the 16 samples as an observation, with a task-clustered bootstrap interval on the difference. Holm correction across these 4 tests leaves both strong-signal comparisons below .05 (resampling $p = 1.2 \times 10^{-8}$, placebo $p = .019$) and neither public-signal comparison below it. N is one lower for the strong-signal resampling row because one localizable candidate has no resampling arm. These pooled tests are sensitivity summaries under both of our analysis plans, not the primary analysis; Table 1 is.

Signal	Comparator	N	Disc.	p	Localized	Comparator	Δpp	95% CI
Public	Random placebo	44	7:1	.070	10.7%	4.5%	+6.1	$[-2.1, +14.9]$
Public	Blind resample	44	2:10	.039	10.7%	12.4%	-1.7	$[-9.8, +7.0]$
Strong	Random placebo	177	11:1	.006	4.3%	1.1%	+3.2	$[+0.5, +6.2]$
Strong	Blind resample	176	3:40	3.0×10^{-9}	4.4%	10.1%	-5.8	$[-9.3, -2.2]$

and, in a later declared replication, a third family. No confirmatory follow-on or positive-mechanism claim is therefore reported.

The workshop supplement ships `analysis_plan.json`, a strict public record of the scientific designations used here. It distinguishes the kickoff unlock rule from the Wave 1B primary outcome: per-attempt success rate for localized versus size-matched placebo under the strong signal, using a task-clustered bootstrap per model and Holm correction across models. The unlock-level test is secondary. Its rationale, that collapsing 16 samples to one bit discards most of the data, is visible in this paper’s own results. Section 3 reports both outcomes, and Table 1 reports the designated primary analysis.

The same public record specifies two additions: a Mistral cross-family replication and a re-run of both Qwen models through the prompted pseudo-FIM path on the tasks they had already run natively. Both are complete. Appendix F reports the interface ablation against its declared interpretation rule, including the selected branch and the measured attenuation.

The workshop supplement maps Table 4 to the per-model and per-benchmark result files behind it, and every reported figure to the statistics summary it is generated from. Generation checkpoints retain the exact localized and placebo spans, which matters because placebo spans were seeded from process-dependent hash values: they are recoverable from the released checkpoints but not reconstructible from task identifiers alone, and the released harness derives seeds from SHA-256 instead. The localizable-task restriction is applied when loading every arm, including resampling; this prevents totals over all dead tasks from being mixed with the smaller paired population.

F The prompted pseudo-FIM interface ablation

Design item (2) in the public analysis record is reported here regardless of its outcome. Every model on this roster without native FIM tokens is also non-Qwen, so family and interface are one partition and neither can be credited with Gemma’s non-replication. Forcing a Qwen down the prompted path varies the interface alone.

Both Qwen models were re-run on the same candidates, the same spans and the same funnel, differing only in how the gap is presented: a chat request to fill a marked hole instead of raw FIM control tokens. The blind-resampling arm is reused from the native run rather than redrawn. It never sees a span and never uses FIM, so the interface cannot reach it, and holding the comparator fixed makes any movement attributable to the interface; all 142 paired candidates carry the identical resampling arm in both conditions. Both conditions in Table 5 are recomputed over these two models and this paired set under a two-model Holm family, so the native rows are not Table 1’s: comparing a two-test family against a three-test family would let a change in multiplicity structure read as an interface effect.

The prompted path is not degenerate here, which is what makes the comparison usable at all: 5.3% of its localized splices fail to parse against 2.5% natively, far from the 65.5% that voided the capability probe, and its no-op rate (49.7%) is essentially the native one. Measured on the localized arm alone, the interface changes little: +2.3 points for Qwen2.5-Coder and -0.5 for Qwen3.6, neither resolving.

Table 5: The interface ablation. Both conditions, both Qwen models, the same 142 paired candidates and the same comparator. Bold marks Holm $p < .05$ within the two-model family.

Condition / model	N	Localized	Δ pp vs resample (95% CI)	Holm p
<i>Native FIM tokens</i>				
Qwen2.5-Coder-32B	91	2.40%	-7.1 [-12.2, -2.2]	.011
Qwen3.6-27B	51	4.90%	-9.1 [-16.3, -2.0]	.015
<i>Prompted pseudo-FIM</i>				
Qwen2.5-Coder-32B	91	4.67%	-4.9 [-9.9, +0.4]	.072
Qwen3.6-27B	51	4.41%	-9.6 [-17.2, -1.7]	.041

We report the attenuation rather than only the branch. Qwen2.5-Coder’s loss narrows enough to stop resolving under correction, on the same candidates and the same N , so the interface is not irrelevant to the size of the effect; what it does not do is reverse the sign in either model or explain why Gemma behaves differently. The placebo comparison under the prompted path resolves in one model, but that is one cell of a four-way exploration this ablation did not designate, and we carry it as exploratory rather than as evidence for a location effect.